

28. Event Delegation

Idea

Bahut saare similar elements ho to **har ek par handler mat lagao**. Unke **common ancestor par ek hi handler** lagao, aur `event.target` se dekho ki asal me kahan event hua.

Example: table cell highlight

```
js

let selectedTd;

table.onclick = function(event) {
  let td = event.target.closest('td'); // (1) sabse paas ka td
  if (!td) return; // (2) td ke andar nahi to ignore
  if (!table.contains(td)) return; // (3) nested table ka td to nahi
  highlight(td); // (4)
};

function highlight(td) {
  if (selectedTd) selectedTd.classList.remove('highlight');
  selectedTd = td;
  selectedTd.classList.add('highlight');
}
```

Kitne bhi `td` hon (ya baad me add/remove ho), code kaam karta hai.

Kyun `closest('td')`? Click `<td>` ke andar ke tag (jaise `<strong>`) par bhi ho sakta hai, tab `event.target` wo inner tag hota hai.

Example: markup me actions

```
html

<div id="menu">
  <button data-action="save">Save</button>
  <button data-action="load">Load</button>
  <button data-action="search">Search</button>
</div>
```

```
js
```

```

class Menu {
  constructor(elem) {
    this._elem = elem;
    elem.onclick = this.onClick.bind(this); // bind zaruri, warna this = DOM e
  }
  save() { alert('saving'); }
  load() { alert('loading'); }
  search() { alert('searching'); }

  onClick(event) {
    let action = event.target.dataset.action;
    if (action) this[action]();
  }
}
new Menu(menu);

```

Har button par alag handler nahi, aur naye buttons kabhi bhi add kar sakte ho.

"Behavior" pattern

Elements me **declaratively** behavior jodo: ek custom attribute lagao + ek document-wide handler.

Counter:

```

html

<input type="button" value="1" data-counter>
<script>
  document.addEventListener('click', function(event) {
    if (event.target.dataset.counter != undefined) event.target.value++;
  });
</script>

```

Toggler:

```

html

```

```
<button data-toggle-id="subscribe-mail">Show form</button>
<form id="subscribe-mail" hidden>...</form>
<script>
  document.addEventListener('click', function(event) {
    let id = event.target.dataset.toggleId;
    if (!id) return;
    let elem = document.getElementById(id);
    elem.hidden = !elem.hidden;
  });
</script>
```

Isse bina JS likhe koi bhi HTML me sirf attribute lagakar behavior add kar sakta hai.

`document` par handler ke liye hamesha `addEventListener` use karo (`document.onclick` se conflict hota hai).

Algorithm

1. Container par ek handler lagao.
2. Handler me `event.target` check karo.
3. Agar kaam ka element hai to handle karo.

Fayde

- Kam initialization aur memory bachat (kam handlers).
- Kam code, elements add/remove karne par handlers jodne/hatane nahi padte.
- `innerHTML` se mass DOM changes kar sakte ho.

Limitations

- Event **bubble** hona chahiye (kuch nahi hote, jaise `focus`). Neeche ke handlers `stopPropagation()` na karein.
- Container handler har event par chalta hai, thoda CPU load (usually negligible).

Tasks (short)

- Message close buttons (`[x]`): ek handler container par, `event.target.closest('.remove-button')` check karke parent pane `remove()`.
- Tree menu, sortable table (`th`) ke `data-type` se sort), tooltip (`mouseover/mouseout` `document` par, `data-tooltip` attribute) sab delegation se bante hain.